BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 10
String Part 2

Table of Contents

SL No	Topic		Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction		-	-	3
	1.1	Objectives	-	-	
2	Operations On Strings		-	-	4-13
	2.1	String Searching And Extraction	-	-	
	2.2	String Case Conversion	-	-	
	2.3	String Modification	-	-	
	2.4	String Formatting: Sprintf() And Snprintf(), Scanf() And Sscanf()	-	1	
3	String Manipulation Programs			2	13-18
4	Summary		-	-	19
5	Terminal Questions		-	-	20
6	Answers To Self Assessment Questions		-	-	20
7	Answers To Terminal Questions		-	-	20

1. INTRODUCTION

Strings serve as fundamental components in programming, offering versatile means for handling textual data. In C programming, understanding various string manipulation techniques is pivotal for effective text processing and data analysis. String searching involves locating specific patterns or substrings within a larger string, essential for tasks like data parsing and pattern matching. Case conversion functions like toupper() and tolower() facilitate altering character cases, ensuring consistency in text processing and simplifying comparison tasks. String modification operations encompass character replacement, whitespace removal, and content insertion, enabling dynamic manipulation of string data. String formatting functions like sprintf() and snprintf() aid in constructing formatted strings from other data types, while scanf() and sscanf() facilitate parsing formatted input strings to extract specific data values. These operations are vital for generating output messages, formatting data for display, and parsing input data effectively. Mastering string manipulation techniques in C programming empowers developers to handle textual data efficiently, enhancing the versatility and robustness of their programs. By understanding string searching, case conversion, modification, and formatting, programmers can implement a wide range of functionalities, from basic text processing tasks to complex data parsing and analysis operations.

1.1. Objectives:

At studying this unit, you should be able to:

- *Understand fundamental string manipulation techniques in C programming for effective text processing and analysis.*
- *learn case conversion functions (toupper() and tolower()) to standardize input data and simplify comparison tasks.*
- *Enhance skills in string modification operations*
- *Utilize string formatting functions (sprintf(), snprintf(), scanf(), sscanf()) for constructing formatted strings and parsing input data accurately.*

2. OPERATIONS ON STRINGS

Operations on strings in C encompass a variety of functionalities vital for manipulating textual data efficiently. These include initialization of strings using character arrays or string literals, input/output operations for reading from and writing to streams, concatenation to merge multiple strings, and length calculation with functions like strlen(). String copying is achieved using strcpy() or strncpy(), while comparison utilizes strcmp(). String searching, extraction, and tokenization involve functions such as strstr(), strchr(), and strtok(). Modification operations include character replacement, whitespace removal, and content insertion. Case conversion alters character cases with functions like toupper() and tolower(). Finally, string formatting constructs formatted strings from other data types using sprintf() and snprintf(), and parses input strings using scanf() and sscanf(). These operations equip programmers with essential tools to effectively handle and process textual data in C programming, facilitating tasks ranging from simple text manipulation to complex data parsing and analysis.

2.1. String Searching and Extraction

String searching and extraction in C refers to the process of locating specific patterns or substrings within a given string and extracting relevant information from it. This involves various operations such as searching for substrings, finding individual characters, tokenizing a string into smaller parts, and extracting substrings based on position and length. These operations are essential for tasks like data parsing, pattern matching, and text analysis in C programming. By utilizing functions and techniques provided by the <string.h> library, programmers can efficiently search for desired patterns, characters, or substrings within text data and extract relevant information for further processing.

Finding a Substring:

To find a substring within a string, we can use functions like strstr() from the standard library. This function searches for the occurrence of a substring within a given string and returns a pointer to the first occurrence if found, or NULL if not found.

```
#include <stdio.h>

#include <string.h>
```

```
int main() {
    char str[] = "hello world";
    char *substr = strstr(str, "world");

    if (substr != NULL) {
        printf("Substring found at index: %ld\n", substr - str);
    } else {
        printf("Substring not found\n");
    }
    return 0;
}
```

This program demonstrates the usage of strstr() function to find a substring ("world") within the string "hello world".

Finding Characters in a String:

We can iterate through each character of a string using a loop and check for specific characters using conditional statements like if. Alternatively, functions like strchr() and strrchr() can be used to find the first or last occurrence of a character in a string, respectively.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "hello world";
    char *ch = strchr(str, 'o');

    if (ch != NULL) {
```

```
        printf("Character found at index: %ld\n", ch - str);
    } else {
        printf("Character not found\n");
    }

    return 0;
}
```

Here, strchr() function is used to find the first occurrence of character 'o' in the string "hello world".

Tokenizing a String:

Tokenizing involves breaking a string into smaller parts, or tokens, based on delimiters such as spaces or punctuation marks. The strtok() function from the standard library can be used to tokenize a string. It returns a pointer to the next token found in the string and modifies the original string to mark the end of the token.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "apple,banana,orange";
    char *token = strtok(str, ",");

    while (token != NULL) {
        printf("%s\n", token);
        token = strtok(NULL, ",");
    }
```

```
    return 0;
}
```

This program tokenizes the string "apple,banana,orange" using strtok() function and prints each token (separated by comma).

Extracting Substrings Based on Position and Length:

We can extract substrings from a string using array indexing or functions like memcpy() or strncpy(). These functions allow you to copy a specified number of characters from one location in memory to another, effectively extracting the desired substring.

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "hello world";
    char substr[6];

    strncpy(substr, str + 6, 5);
    substr[5] = '\0';

    printf("Substring: %s\n", substr);

    return 0;
}
```

In this example, strncpy() function is used to extract a substring of length 5 starting from index 6 of the string "hello world".

2.2. String Case Conversion

String case conversion in C refers to the process of changing the case of characters within a string. This typically involves converting all characters to uppercase or lowercase. In C programming, you can achieve string case conversion using functions like toupper() and tolower() provided by the <ctype.h> header file.

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>

int main() {

   char str[] = "Hello World";


   // Convert all characters to uppercase

   for (int i = 0; i < strlen(str); i++) {

      str[i] = toupper(str[i]);

   }

   printf("Uppercase string: %s\n", str);


   // Convert all characters to lowercase

   for (int i = 0; i < strlen(str); i++) {

      str[i] = tolower(str[i]);

   }

   printf("Lowercase string: %s\n", str);


   return 0;

}
```

This program demonstrates how to convert a string to uppercase and then to lowercase using toupper() and tolower() functions. These functions take a character as input and return the corresponding uppercase or lowercase equivalent. By iterating through each character of the string, you can apply these functions to perform case conversion.

2.3. String Modification

String modification in C involves altering the content of a string. This can include tasks such as replacing characters, removing whitespace, inserting additional content, or making other changes as required by the program.

```
#include <stdio.h>

#include <string.h>


int main() {

   char str[] = "Hello, World!";


   // Replace a character

   str[7] = 'P'; // Replace 'W' with 'P'


   // Remove whitespace

   int src = 0, dst = 0;

   while (str[src]) {

      if (str[src] != ' ') {

         str[dst++] = str[src];

      }

      src++;

   }
```

```
    str[dst] = '\0'; // Null-terminate the modified string


    // Insert additional content
    strcat(str, " Welcome");


    printf("Modified string: %s\n", str);


    return 0;
}
```

In this C Program:

We replace the character at index 7 with 'P'.

We remove all whitespace characters from the string.

We insert the string " Welcome" at the end using the strcat() function.

This demonstrates how to perform various string modification operations in C to achieve the desired changes in the string. Depending on the specific requirements, you may need to use different techniques and functions for string modification.

2.4. String Formatting: sprintf() and snprintf(), scanf() and sscanf()

String formatting in C involves constructing formatted strings from other data types and parsing formatted input strings to extract specific data values. This is commonly done using functions like sprintf(), snprintf(), scanf(), and sscanf().

sprintf(): This function formats and stores a series of characters and values in a string. It is similar to printf(), but instead of printing to the standard output, it writes to a string.

snprintf(): Similar to sprintf(), snprintf() formats and stores characters and values in a string, but with additional functionality to specify the maximum number of characters to be written.

scanf(): This function reads formatted input from the standard input stream (usually the keyboard) and assigns the values to variables based on the specified format string.

sscanf(): Similar to scanf(), sscanf() reads formatted input from a string instead of the standard input stream. It parses the input string according to the specified format and assigns the extracted values to variables.void insertionSort(int arr[], int n)

```
#include <stdio.h>

int main() {

   char str[100];

   int num1 = 10, num2 = 20;


   // Using sprintf() to format a string

   sprintf(str, "Sum of %d and %d is %d", num1, num2, num1 + num2);

   printf("Formatted string: %s\n", str);


   // Using snprintf() to avoid buffer overflow

   snprintf(str, sizeof(str), "Sum of %d and %d is %d", num1, num2, num1 + num2);

   printf("Formatted string: %s\n", str);


   // Using scanf() to read formatted input

   int input;

   printf("Enter a number: ");

   scanf("%d", &input);

   printf("You entered: %d\n", input);


   // Using sscanf() to parse formatted input from a string
```

```
    char data[] = "Name: John Age: 25";
    char name[20];
    int age;
    sscanf(data, "Name: %s Age: %d", name, &age);
    printf("Name: %s, Age: %d\n", name, age);

    return 0;
}
```

This c program showcases the basic usage of sprintf(), snprintf(), scanf(), and sscanf() for string formatting and parsing in C programming.

SELF-ASSESSMENT QUESTIONS - 1

1. What function is used to read formatted input from a string in C?
   (a) sscanf()
   (b) scanf()
   (c) fscanf()
   (d) sprint()
2. Which function is used to tokenize a string into smaller strings based on delimiters in C?
   (a) strsep()
   (b) strtok()
   (c) strspn()
   (d) strcspn()

3. Which function is used to find the first occurrence of a substring within a string in C?
   (e) strstr()
   (f) strchr()
   (g) strtok()
   (h) strcspn()

3. STRING MANIPULATION PROGRAMS

To facilitate string manipulation, C provides a range of functions in the standard library "string.h". These functions offer efficient solutions to common tasks, reducing the complexity and size of programs.

Here are a few commonly used string handling functions:

Operation	**Function**
String Length Calculation	**strlen()**
String Concatenation	**strcat()**
String Copy	**strcpy()**
String Comparison	**strcmp()**
Finding Substring	**strstr()**
Case Conversion	**toupper()** and **tolower()** for individual characters, custom implementation for whole string

These string handling functions are declared in the "string.h" header file.

Here are the string manipulation operations implemented in C:

String Length Calculation: To calculate the length of a string in C, you can use the strlen() function from the <string.h> header**.**

```
#include <stdio.h>
#include <string.h>
int main() {
   char str[] = "Hello, World!";
   int length = strlen(str);
   printf("Length of the string: %d\n", length);
   return 0;
}
```

String Concatenation: To concatenate two strings in C, you can use the strcat() function from the <string.h> header. Here is an example:

```
#include <stdio.h>
#include <string.h>

int main() {
   char str1[20] = "Hello";
   char str2[] = ", World!";
   strcat(str1, str2);
   printf("Concatenated string: %s\n", str1);
   return 0;
}
```

String Copy: To copy one string to another in C, you can use the strcpy() function from the <string.h> header.

```
#include <stdio.h>
#include <string.h>
int main() {
   char src[] = "Hello, World!";
   char dest[20];
   strcpy(dest, src);
   printf("Copied string: %s\n", dest);
   return 0;
}
```

String Comparison: To compare two strings in C, you can use the strcmp() function from the <string.h> header.

```
#include <stdio.h>
#include <string.h>
int main() {
   char str1[] = "hello";
   char str2[] = "world";
   int result = strcmp(str1, str2);
   if (result == 0) {
      printf("Strings are equal\n");
   } else {
      printf("Strings are not equal\n");
   }
   return 0;
```

```
}
```

String Reversal:

```
#include <stdio.h>
#include <string.h>

int main() {
   char str[] = "Hello, World!";
   int length = strlen(str);
   for (int i = length - 1; i >= 0; i--) {
      printf("%c", str[i]);
   }
   printf("\n");
   return 0;
}
```

The code reverses the string "Hello, World!" by iterating backward through its characters and printing them, resulting in the reversed string "dlroW ,olleH".

Substring Extraction:

```
#include <stdio.h>
#include <string.h>

int main() {
   char str[] = "Hello, World!";
   char sub[20];
   strncpy(sub, str + 7, 5);
```

```
    sub[5] = '\0';
    printf("Substring: %s\n", sub);
    return 0;
}
```

Finding Substring: To find a substring within a string in C, you can use the strstr() function from the <string.h> header.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello, World!";
    char *substr = strstr(str, "World");
    if (substr != NULL) {
        printf("Substring found at index: %ld\n", substr - str);
    } else {
        printf("Substring not found\n");
    }
    return 0;
}
```

Case Conversion: To convert the case of characters in a string in C, we can use the standard library functions toupper() and tolower().

```
#include <stdio.h>
#include <ctype.h>

int main() {
```

```
    char str[] = "Hello, World!";
    for (int i = 0; str[i] != '\0'; i++) {
        str[i] = toupper(str[i]);
    }
    printf("Uppercase string: %s\n", str);
    return 0;
}
```

These programs demonstrate various string manipulation operations in C.

SELF-ASSESSMENT QUESTIONS – 2

4. Which function is used for calculating the length of a string in C?
   (a) strlength()
   (b) lengthstr()
   (c) strlen()
   (d) strcount()
5. Which function is used for extracting a substring from a string in C?
   (a) subextract()
   (b) strsubstr()
   (c) substr()
   (d) strstr()
6. Which functions are used for case conversion of characters in C?
   (a) toupper() and tolower()
   (b) convertcase() and casechange()
   (c) changecase() and casemanipulate()
   (d) switchcase() and transformcase()

4. SUMMARY

String searching and extraction tasks involve finding substrings within strings, locating specific characters, tokenizing strings based on delimiters, and extracting substrings based on specified positions and lengths. String case conversion functions, including toupper() and tolower(), enable the transformation of characters to uppercase and lowercase, respectively, facilitating versatile text processing. String modification operations encompass various tasks such as appending, replacing, or removing characters, allowing for flexible string manipulation. String formatting functions like sprintf() and snprintf() facilitate precise string representation, while scanf() and sscanf() enable reading formatted input from strings, enhancing input processing capabilities. Additional string operations include determining string lengths without relying on library functions, concatenating strings, copying string contents, comparing strings, reversing string characters, extracting substrings based on specified indices, and finding substrings within strings. Mastering these operations equips C programmers with essential skills for developing efficient and reliable software solutions across diverse domains, enhancing code readability, efficiency, and functionality. Understanding and effectively utilizing these string operations are crucial for developing robust software solutions that meet specific project requirements.

5. TERMINAL QUESTIONS

1. Explain the difference between strchr() and strstr() functions in C with respect to string searching.
2. Explain the purpose of the toupper() and tolower() functions in C. Provide an example.
3. Describe the process of modifying a string in C. Provide an example demonstrating a modification operation.

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. (a) sscanf()
2. (b) strtok()
3. (a) strstr()
4. (c) strlen()
5. (c) substr()
6. (a) toupper() and tolower()

7. ANSWERS TO TERMINAL QUESTIONS

1. (Refer Section 2 for more details)
2. (Refer Section 3 for more details)
3. (Refer Section 2 for more details)